WEB APPLICATION SECURITY TESTING PORTFOLIO

Selected Manual Findings, Validation Methodology & Security Reporting Capability

Candidate	Harshit Ojha
Profile	Cybersecurity Fresher | VAPT / Web Application Security
Project type	Authorized lab-based security assessment - not employment or commercial client experience
Testing approach	Manual validation + Burp Suite Professional automated assessment
Focus	OWASP-aligned web vulnerabilities, CWE mapping, evidence, risk, remediation and retest

Purpose of this portfolio

This document is designed for recruiters and technical interviewers. It demonstrates how I approach an authorized web application assessment: identify a weakness, reproduce it manually, distinguish scanner output from confirmed risk, explain business impact, recommend a fix, and define how the fix should be retested.

RECRUITER VIEW

Capability Snapshot

This portfolio intentionally presents the work as a hands-on authorized lab project. It does not claim prior company employment, client ownership, production access, or commercial consulting experience. The objective is to show practical security-testing and reporting capability.

Finding	CWE	OWASP theme	Validation
SQL Injection	CWE-89	A03: Injection	Manual + Automated
IDOR / Broken Object Authorization	CWE-639	A01: Broken Access Control	Manual
Session Fixation	CWE-384	A07: Identification & Authentication Failures	Manual
Cleartext Credential Transmission	CWE-319	A02: Cryptographic Failures	Manual
Session Cookie - Secure flag missing	CWE-614	A07 / session hardening	Manual

Reporting approach

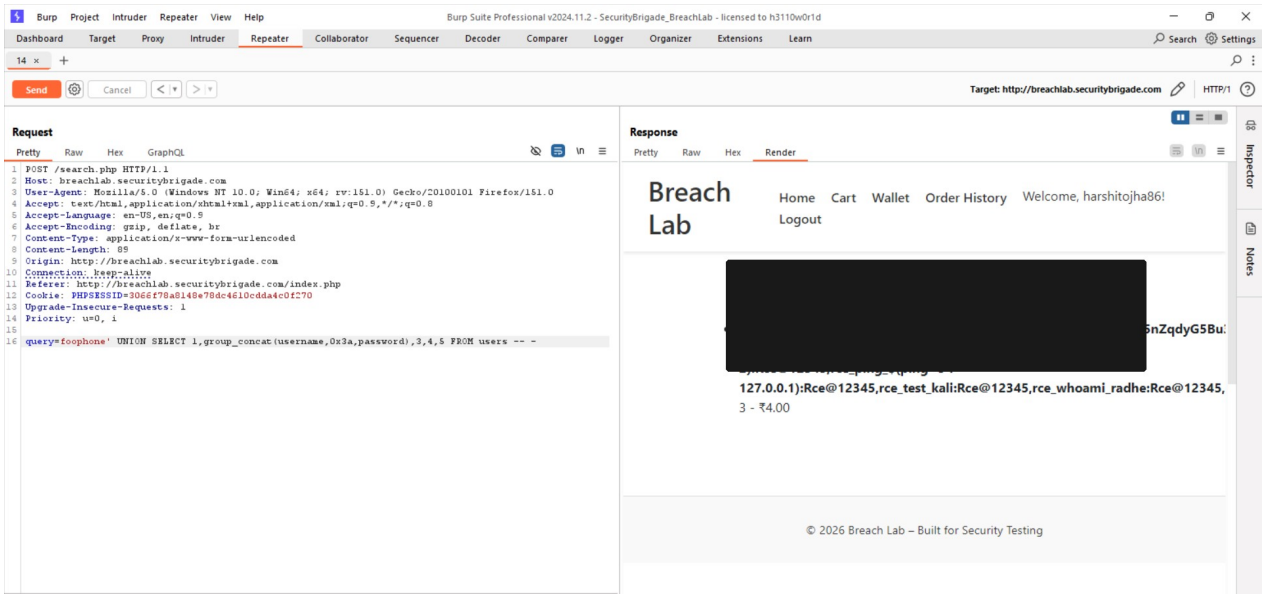
Confirmed findings are separated from tentative scanner observations. Duplicate manual and automated detections are consolidated into one root-cause finding instead of inflating the vulnerability count.

HIGH-IMPACT FINDING

01 - SQL Injection

Affected area: Product search | Endpoint: POST /search.php | Parameter: query | CWE: CWE-89

How I validated it. I intercepted the search request, sent it to Burp Repeater, modified the query input, and observed that crafted input influenced backend database behavior. Manual validation demonstrated database-derived credential material in the response. Burp Scanner independently flagged the same parameter with High / Firm confidence and time-delay behavior.



Manual proof - database-derived output returned after crafted input. Sensitive credential material has been redacted for portfolio use.

Security impact

Unsafe SQL construction can expose or modify application data and undermine application trust. The exact blast radius depends on database privileges and application architecture.

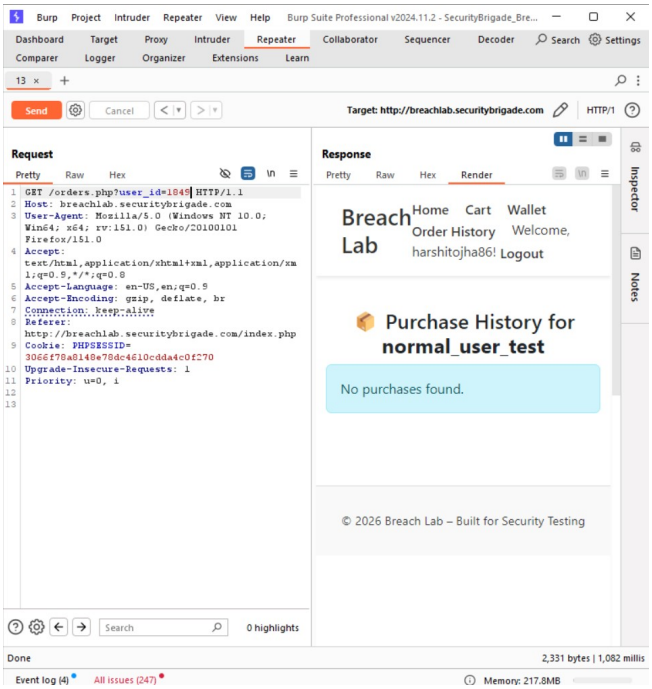
Recommended remediation: use parameterized queries / prepared statements for all database access; apply server-side validation as defense in depth; use least-privilege database accounts; avoid exposing database errors; and regression-test all database-backed inputs.

Retest: the original input and controlled boolean/time-delay cases must be treated strictly as data, with no database-derived leakage, error behavior, or database-controlled delay.

02 - IDOR / Broken Object Level Authorization

Affected area: Order History | Endpoint: GET /orders.php | Parameter: user_id | CWE: CWE-639

How I validated it. While authenticated as one account, I changed the user-controlled identifier in the order-history request. The application returned purchase-history context for a different account, demonstrating that authentication alone was being trusted without sufficient object-level authorization.



Manual proof - the modified user_id returns order-history context for a different account while the original authenticated session remains active.

Security impact

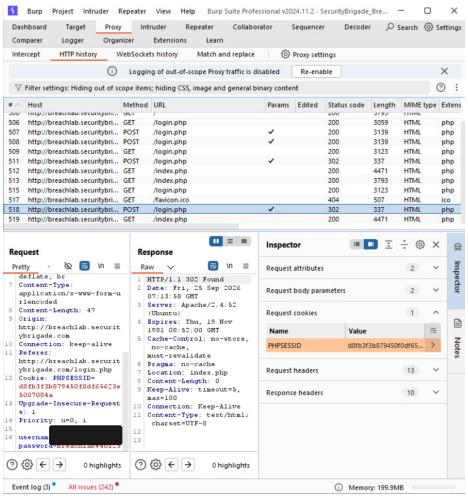
In a real application, broken object authorization can expose another customer's protected records and create confidentiality, privacy and compliance risk.

Recommended remediation: enforce server-side authorization for every object request. Derive accessible objects from the authenticated identity wherever possible; otherwise verify ownership/permission before returning data. Random or UUID identifiers can reduce guessability but do not replace authorization.

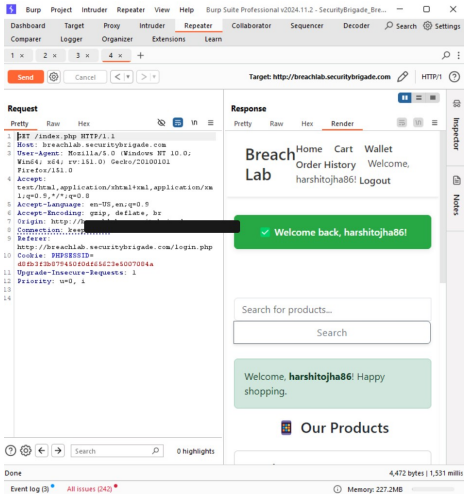
03 - Session Fixation

Affected control: PHPSESSID lifecycle | CWE: CWE-384

How I validated it. I recorded the session identifier before login, authenticated successfully, and verified that the same identifier remained valid for authenticated content. The important observation is the missing trust-boundary rotation from anonymous to authenticated state.



Step 1 - pre-authentication session persists through successful login. Password value is redacted.



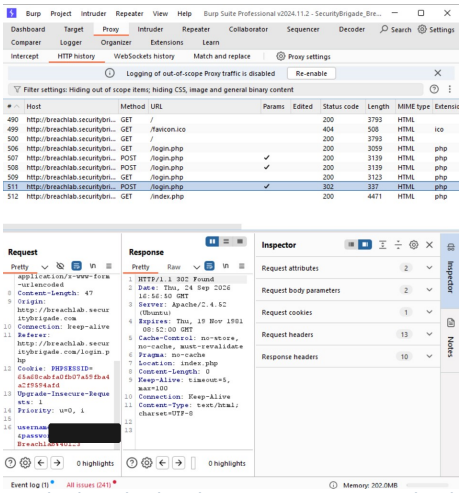
Step 2 - the retained session is accepted for authenticated content. Session-token material is partially redacted in the portfolio.

Recommended remediation: regenerate the session identifier immediately after authentication and privilege changes, invalidate the old identifier server-side, use strict session handling, and implement secure logout and timeout behavior.

04 - Transport & Session Cookie Security

Finding A: Cleartext Transmission of Login Credentials over HTTP - CWE-319

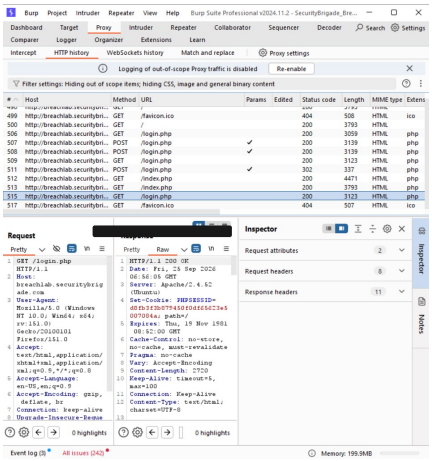
The issue is not that a password is visible inside Burp; an intercepting proxy can show decrypted HTTPS requests too. The security issue is that the login transaction itself used HTTP without TLS, allowing suitable network-positioned attackers or intermediaries to observe or alter sensitive traffic.



Manual evidence - login submitted over HTTP. Password value is redacted.

Finding B: Session Cookie Missing Secure Attribute - CWE-614

The observed PHPSESSID cookie was issued without the Secure attribute. The captured configuration also lacked HttpOnly and an explicit SameSite attribute; these are treated as related cookie-hardening observations rather than proof of XSS or CSRF.

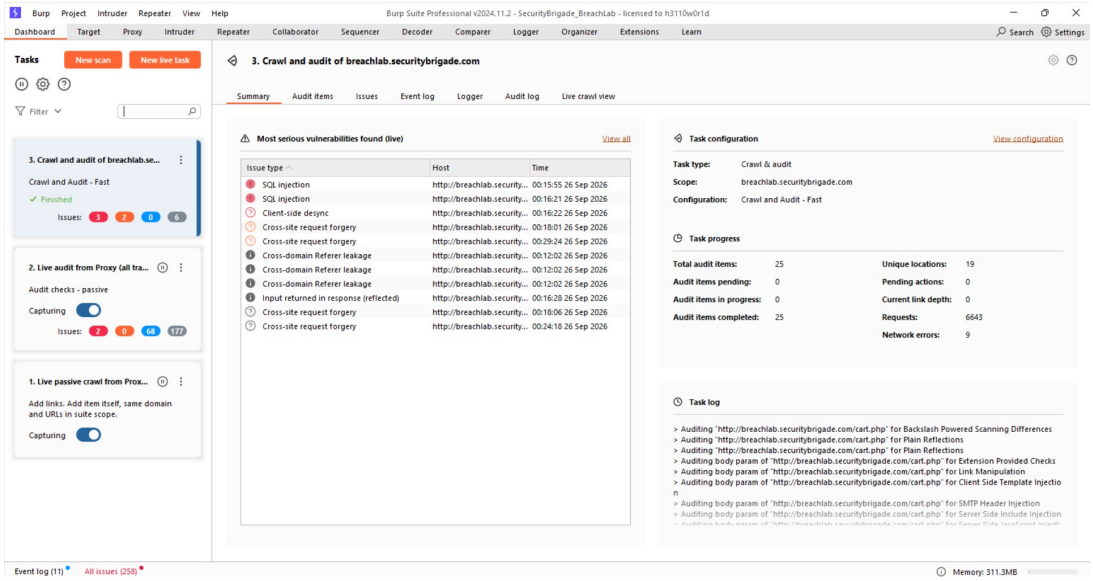


Manual evidence - session cookie configuration. Token material is redacted for portfolio sharing.

Recommended remediation: enforce HTTPS application-wide, redirect or disable plaintext HTTP, deploy HSTS when appropriate, set session cookies Secure and HttpOnly, and choose a SameSite policy compatible with legitimate application flows.

Manual Validation vs Automated Scanning

Automated scanners are valuable for breadth, but I do not treat every scanner alert as a confirmed vulnerability. My workflow is: discover -> reproduce -> compare behavior -> establish security impact -> document -> recommend -> retest.



Burp automated assessment used as supporting coverage. Scanner findings were triaged rather than copied directly into the final confirmed list.

Automated observation	Treatment
SQL Injection - /search.php query	Manually confirmed and consolidated with the manual finding.
SQL Injection - /cart.php cart_id	Strong automated candidate; requires manual reproduction before claiming confirmed.
Client-side desync - /index.php	Scanner confidence was tentative; not promoted without focused validation.
CSRF candidates	Not promoted merely because a token was absent; meaningful state change and cross-site exploitability must be established.
Reflected input	Informational observation; reflection alone is not XSS.

Security Reporting & GRC Perspective

For each confirmed finding, I structure reporting so that both technical and non-technical stakeholders can act on it. The report links evidence to a root cause and a closure condition rather than only presenting payloads or scanner severity.

Report element	Why it matters
Executive summary	Allows management to understand exposure and priorities without reading raw requests.
CWE / OWASP mapping	Provides a common security taxonomy and helps organize recurring control weaknesses.
Affected endpoint & parameter	Gives engineering a precise remediation target.
Business + technical impact	Connects exploit behavior to organizational risk without overstating evidence.
Remediation	Defines the root-cause fix, not just a payload blacklist or WAF rule.
Retest acceptance criteria	Makes closure measurable and supports audit / remediation tracking.
Scanner triage	Reduces false positives and avoids duplicate vulnerability counts.

Candidate positioning

This portfolio demonstrates hands-on lab capability and security-consulting style documentation. It should be evaluated as project evidence for an entry-level role, not represented as previous company employment or production-client experience.

INTERVIEW HANDOFF

Portfolio Summary

What I can demonstrate: manual HTTP request analysis, Burp Repeater validation, parameter tampering, access-control testing, authentication/session analysis, SQL injection validation, scanner-result triage, CWE/OWASP mapping, business-impact writing, remediation design and retest planning.

What I would do next in a real engagement: follow the agreed scope and rules of engagement, maintain evidence hygiene, validate high-impact findings with the minimum necessary proof, avoid unnecessary data access, communicate critical issues promptly, and perform a structured retest after remediation.

One-line portfolio statement

Performed an authorized web application security lab assessment, manually validated multiple OWASP-aligned vulnerabilities, correlated automated findings, and produced a risk- and remediation-focused security report suitable for technical and management review.

Harshit Ojha
Cybersecurity Fresher | VAPT / Web Application Security
Portfolio prepared September 2026